

LegalEase AI

Project Source Code & Configuration Document

BACKEND

GEMINI GENERATOR.PY

```
import os
from google import genai
class GeminiDocumentGenerator:
    def __init__(self, api_key=None, model=None):
        self.api_key = api_key or os.getenv("GEMINI_API_KEY")
        if not self.api_key:
            raise ValueError("GEMINI_API_KEY is not configured.")
        self.model = model or "gemini-3.6-flash"
        self.client = genai.Client(
            api_key=self.api_key
        )
    def generate_document(
        self,
        document_type,
        parties,
        terms,
        effective_date
    ):
        prompt = f"""
Create a professional first-draft legal document.
Document Type:
{document_type}
Parties:
{parties}
Terms and Conditions:
{terms}
Effective Date:
{effective_date}
Requirements:
- Create a clear professional legal document.
- Use suitable headings and sections.
- Include all parties.
- Include the effective date.
- Include all provided terms.
- Add signature sections where appropriate.
- Do not invent important facts.
- This is a first draft and must be reviewed before use.
Return only the legal document.
"""
        response = self.client.models.generate_content(
            model=self.model,
            contents=prompt
        )
        return response.text
```

```

MAIN.PY
import os
from dotenv import load_dotenv
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from backend.routes import router
load_dotenv()
app = FastAPI(
    title="LegalEase API",
    description="AI-powered legal document draft generator",
    version="1.0.0",
)
origins = [
    item.strip()
    for item in os.getenv(
        "CORS_ORIGINS",
        "http://localhost:8501,http://127.0.0.1:8501",
    ).split(",")
    if item.strip()
]
app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=False,
    allow_methods=["GET", "POST"],
    allow_headers=["*"],
)
app.include_router(
    router,
    prefix="/api"
)
@app.get("/")
def root():
    return {
        "message": "LegalEase API is running",
        "docs": "/docs"
    }
@app.get("/health")
def health():
    return {
        "status": "ok"
    }

```

```

ROUTES.PY
import os
from fastapi import APIRouter, HTTPException
from ai_core.gemini_generator import GeminiDocumentGenerator
from backend.schemas import (
    DocumentRequest,
    DocumentResponse,
)
router = APIRouter(
    tags=["documents"]
)
@router.post(

```

```

        "/generate",
        response_model=DocumentResponse
    )
    def generate_document(
        request: DocumentRequest
    ):
        try:
            generator = GeminiDocumentGenerator(
                api_key=os.getenv("GEMINI_API_KEY"),
                model=os.getenv(
                    "GEMINI_MODEL",

```

gemini- 3.8- flash

```

    )
    )
    content = generator.generate_document(
        document_type=request.document_type,
        parties=request.parties,
        terms=request.terms,
        effective_date=request.effective_date,
    )
    return DocumentResponse(
        document_type=request.document_type,
        content=content,
        model=generator.model,
    )
except ValueError as exc:
    raise HTTPException(
        status_code=400,
        detail=str(exc)
    ) from exc
except Exception as exc:
    raise HTTPException(
        status_code=502,
        detail=f"AI generation failed: {exc}"
    ) from exc
SCHEMAS.PY
from pydantic import BaseModel, Field, field_validator
class DocumentRequest(BaseModel):
    document_type: str = Field(
        ...,
        min_length=2,
        max_length=120
    )
    parties: str = Field(
        ...,
        min_length=3,
        max_length=5000
    )
    terms: str = Field(
        ...,
        min_length=3,
        max_length=10000

```

```

)
effective_date: str = Field(
    ...,
    min_length=1,
    max_length=100
)
@field_validator(
    "document_type",
    "parties",
    "terms",

```

effective_date

```

)
@classmethod
def validate_text(cls, value: str) -> str:
    value = value.strip()
    if not value:
        raise ValueError(

```

This field cannot be empty.

```

    )
    return value
class DocumentResponse(BaseModel):
    document_type: str
    content: str
    model: str

```

FRONTEND

```

APP.PY
import os
from datetime import date
import requests
import streamlit as st
from dotenv import load_dotenv
from services.document_formatter import (
    format_docx,
    format_html_preview,
    format_pdf,
    format_txt,
    sanitize_text,
)
# =====
# ENVIRONMENT
# =====
load_dotenv()
BACKEND_URL = os.getenv(
    "BACKEND_URL",
)

http://127.0.0.1:8000

).rstrip("/")
# =====

```

```

# PAGE CONFIG
# =====
st.set_page_config(
    page_title="LegalEase AI",
    page_icon=" ",
    layout="wide",
)
# =====
# CUSTOM CSS
# =====
st.markdown(
    """
    <style>
    .main-title {
        text-align: center;
        font-size: 42px;
        font-weight: 800;
        margin-top: 10px;
        margin-bottom: 5px;
    }
    .subtitle {
        text-align: center;
        font-size: 18px;
        opacity: 0.75;
        margin-bottom: 30px;
    }
    .info-box {
        padding: 18px;
        border-radius: 12px;
        border: 1px solid rgba(128,128,128,0.3);
        margin-bottom: 20px;
    }
    .legal-preview {
        padding: 25px;
        border-radius: 12px;
        border: 1px solid #444;
        line-height: 1.7;
        font-family: Georgia, "Times New Roman", serif;
        max-height: 650px;
        overflow-y: auto;
    }
    </style>
    """
    ,
    unsafe_allow_html=True,
)
# =====
# HEADER
# =====
st.markdown(
    '<div class="main-title"> LegalEase AI</div>',
    unsafe_allow_html=True,
)
st.markdown(
    '<div class="subtitle">AI-Powered Legal Document Generator</div>',

```

```

        unsafe_allow_html=True,
    )
    # =====
    # SIDEBAR
    # =====
    with st.sidebar:
        st.header(" Settings")
        st.write("Backend URL")
        st.code(
            BACKEND_URL,
            language="text",
        )
        st.divider()
        if st.button(
            " Check Backend",
            use_container_width=True,
        ):
            try:
                response = requests.get(
                    f"{BACKEND_URL}/health",
                    timeout=5,
                )
                if response.ok:
                    st.success(

```

Backend is online

```

        )
    else:
        st.error(
            f"Backend returned HTTP {response.status_code}"
        )
    except requests.RequestException as exc:
        st.error(

```

Backend is not reachable.

```

        )
        st.caption(
            str(exc)
        )
    # =====
    # INFORMATION
    # =====
    st.info(
        """
        **LegalEase AI**
        Generate editable first drafts of legal documents using AI.
        Please review generated documents carefully for applicable
        laws, regulations, facts, and legal requirements before
        signing or relying on them.
        """
    )
    # =====

```

```
# DOCUMENT INPUT SECTION
```

```
# =====
```

```
st.header(" Create Your Legal Document")
```

```
# -----
```

```
# DOCUMENT TYPE
```

```
# -----
```

```
document_type = st.selectbox(
```

```
    "Document Type",
```

```
    [
```

```
        "Employment Contract",
```

```
        "Non-Disclosure Agreement (NDA)",
```

```
        "Lease Agreement",
```

```
        "Freelance Work Contract",
```

```
        "Service Agreement",
```

```
        "General Agreement",
```

```
        "Custom Legal Document",
```

```
    ],
```

```
)
```

```
# -----
```

```
# PARTIES
```

```
# -----
```

```
parties = st.text_area(
```

```
    " Parties Involved",
```

```
    placeholder=(
```

Example:\n

Jane Doe (Service Provider)\n

ABC Technologies Pvt. Ltd. (Client)

```
),
```

```
height=130,
```

```
)
```

```
# -----
```

```
# TERMS
```

```
# -----
```

```
terms = st.text_area(
```

```
    " Terms & Conditions",
```

```
    placeholder=(
```

Example:\n

Payment within 30 days;

Confidential information must not be disclosed;

Either party may terminate with 30 days notice

```
),
```

```
height=180,
```

```
)
```

```
# -----
```

```

# EFFECTIVE DATE
# -----
effective_date = st.date_input(
    " Effective Date",
    value=date.today(),
)
# =====
# SESSION STATE
# =====
if "generated_text" not in st.session_state:
    st.session_state.generated_text = ""
if "generated_model" not in st.session_state:
    st.session_state.generated_model = ""
# =====
# GENERATE BUTTON
# =====
st.divider()
generate_clicked = st.button(
    " Generate Legal Document",
    type="primary",
    use_container_width=True,
)
if generate_clicked:
    # -----
    # VALIDATION
    # -----
    if not parties.strip():
        st.warning(

```

Please enter the parties involved.

```

)
elif not terms.strip():
    st.warning(

```

Please enter the terms and conditions.

```

)
else:
    payload = {
        "document_type": document_type,
        "parties": parties.strip(),
        "terms": terms.strip(),
        "effective_date": effective_date.isoformat(),
    }
    # -----
    # API REQUEST
    # -----
    with st.spinner(

```

AI is generating your legal document...

```

):
    try:
        response = requests.post(

```

```

    f"{BACKEND_URL}/api/generate",
    json=payload,
    timeout=180,
)
# -----
# SUCCESS
# -----
if response.status_code == 200:
    data = response.json()
    generated_content = data.get(
        "content",
        "",
    )
    if not generated_content:
        st.error(

```

Backend returned an empty document.

```

    )
else:
    st.session_state.generated_text = (
        sanitize_text(
            generated_content
        )
    )
    st.session_state.generated_model = (
        data.get(
            "model",
            "AI model",
        )
    )
    st.success(

```

Document generated successfully!

```

    )
# -----
# ERROR
# -----
else:
    try:
        error_data = response.json()
        error_detail = error_data.get(
            "detail",
            f"HTTP {response.status_code}",
        )
    except Exception:
        error_detail = (
            f"HTTP {response.status_code}: "
            f"{response.text}"
        )
    st.error(
        f" {error_detail}"
    )

```

```
except requests.Timeout:
    st.error(
```

Request timed out.

The AI service took too long to respond.

```
)
except requests.ConnectionError:
    st.error(
```

Could not connect to the FastAPI backend.

```
)
    st.info(
```

Make sure the backend is running on

```
    f"{BACKEND_URL}"
)
except requests.RequestException as exc:
    st.error(
        f" Request failed: {exc}"
    )
except Exception as exc:
    st.error(
        f" Unexpected error: {exc}"
    )
# =====
# GENERATED DOCUMENT
# =====
if st.session_state.generated_text:
    st.divider()
    st.header(" Generated Document")
    # -----
    # MODEL INFORMATION
    # -----
    if st.session_state.generated_model:
        st.caption(
            f"Generated using: "
            f"{st.session_state.generated_model}"
        )
    # -----
    # EDITOR
    # -----
    edited_text = st.text_area(
        "Edit your document if required",
        value=st.session_state.generated_text,
        height=550,
        key="document_editor",
    )
    # Update session state
    st.session_state.generated_text = edited_text
# =====
```

```

# PREVIEW
# =====
st.subheader(" Document Preview")
try:
    preview_html = format_html_preview(
        edited_text
    )
    st.markdown(
        preview_html,
        unsafe_allow_html=True,
    )
except Exception as exc:
    st.warning(
        f"Preview could not be generated: {exc}"
    )
    st.text(
        edited_text
    )
# =====
# DOWNLOAD SECTION
# =====
st.subheader(" Download Document")
safe_name = (
    document_type
    .lower()
    .replace(" ", "_")
    .replace("(", "")
    .replace(")", "")
    .replace("/", "_")
    .replace("\\", "_")
)
col1, col2, col3 = st.columns(3)
# -----
# TXT
# -----
with col1:
    try:
        txt_data = format_txt(
            edited_text
        )
        st.download_button(
            label=" Download TXT",
            data=txt_data,
            file_name=f"{safe_name}.txt",
            mime="text/plain",
            use_container_width=True,
        )
    except Exception as exc:
        st.error(
            f"TXT export failed: {exc}"
        )
# -----
# DOCX
# -----

```

```

with col2:
    try:
        docx_data = format_docx(
            edited_text,
            document_type,
        )
        st.download_button(
            label=" Download DOCX",
            data=docx_data,
            file_name=f"{safe_name}.docx",
            mime=(

```

application/

vnd.openxmlformats-officedocument.

wordprocessingml.document

```

        ),
        use_container_width=True,
    )
except Exception as exc:
    st.error(
        f"DOCX export failed: {exc}"
    )
# -----
# PDF
# -----
with col3:
    try:
        pdf_data = format_pdf(
            edited_text,
            document_type,
        )
        st.download_button(
            label=" Download PDF",
            data=pdf_data,
            file_name=f"{safe_name}.pdf",
            mime="application/pdf",
            use_container_width=True,
        )
    except Exception as exc:
        st.error(
            f"PDF export failed: {exc}"
        )
# =====
# FOOTER
# =====
st.divider()
st.caption(

```

LegalEase AI • AI- assisted legal document drafting •

Review all generated documents before use.

```
)
TEST_APP.PY
import streamlit as st
st.set_page_config(
    page_title="LegalEase Test",
    page_icon=" "
)
st.title(" LegalEase Test")
st.success("Streamlit is working!")
st.write("If you can see this message, the Streamlit frontend is working correctly.")
```

SERVICES

```
DOCUMENT_FORMATTER.PY
from __future__ import annotations
import html
import io
import re
from xml.sax.saxutils import escape
from docx import Document
from docx.shared import Pt
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.enums import TA_CENTER
from reportlab.platypus import (
    SimpleDocTemplate,
    Paragraph,
    Spacer,
)
# -----
# TEXT SANITIZATION
# -----
def sanitize_text(text: str) -> str:
    """
    Clean AI-generated text before displaying/exporting it.
    """
    if text is None:
        return ""
    text = str(text)
    # Normalize line endings
    text = text.replace("\r\n", "\n")
    text = text.replace("\r", "\n")
    # Remove accidental markdown code fences
    text = re.sub(r"```(?:text|markdown)?", "", text)
    text = text.replace("```", "")
    # Remove excessive blank lines
    text = re.sub(r"\n{3,}", "\n\n", text)
    return text.strip()
# -----
# HTML PREVIEW
# -----
def format_html_preview(text: str) -> str:
```

```
"""
Convert document text into safe HTML for Streamlit preview.
"""
```

```
text = sanitize_text(text)
if not text:
    return "<p>No document content available.</p>"
escaped = html.escape(text)
# Convert blank lines to paragraphs
paragraphs = re.split(r"\n\s*\n", escaped)
html_parts = []
for paragraph in paragraphs:
    paragraph = paragraph.strip()
    if not paragraph:
        continue
    # Preserve single line breaks
    paragraph = paragraph.replace("\n", "<br>")
    html_parts.append(
        f"<p style='margin-bottom: 14px;'>{paragraph}</p>"
    )
return f"""
<div class="legal-preview">
    {" ".join(html_parts)}
</div>
"""
```

```
# -----
# TXT FORMAT
# -----
```

```
def format_txt(text: str) -> bytes:
    """
```

```
    Create a UTF-8 TXT file.
    """
```

```
    clean_text = sanitize_text(text)
    return clean_text.encode("utf-8")
```

```
# -----
# DOCX FORMAT
# -----
```

```
def format_docx(
    text: str,
    document_type: str = "Legal Document",
) -> bytes:
    """
```

```
    Create a DOCX document from generated legal text.
    """
```

```
    clean_text = sanitize_text(text)
    document = Document()
    # -----
    # Default font
    # -----
    styles = document.styles
    normal_style = styles["Normal"]
    normal_style.font.name = "Times New Roman"
    normal_style.font.size = Pt(12)
    # -----
```

```
    # Title
```

```

# -----
title = document.add_paragraph()
title.alignment = 1
title_run = title.add_run(
    document_type.upper()
)
title_run.bold = True
title_run.font.name = "Times New Roman"
title_run.font.size = Pt(16)
document.add_paragraph()
# -----
# Content
# -----
lines = clean_text.split("\n")
for line in lines:
    line = line.strip()
    if not line:
        document.add_paragraph()
        continue
    paragraph = document.add_paragraph()
    run = paragraph.add_run(line)
    run.font.name = "Times New Roman"
    run.font.size = Pt(12)
# -----
# Save to memory
# -----
output = io.BytesIO()
document.save(output)
output.seek(0)
return output.getvalue()
# -----
# PDF FORMAT
# -----
def format_pdf(
    text: str,
    document_type: str = "Legal Document",
) -> bytes:
    """
    Create a PDF document from generated legal text.
    """
    clean_text = sanitize_text(text)
    output = io.BytesIO()
    pdf = SimpleDocTemplate(
        output,
        pagesize=A4,
        rightMargin=50,
        leftMargin=50,
        topMargin=50,
        bottomMargin=50,
        title=document_type,
    )
    styles = getSampleStyleSheet()
    title_style = ParagraphStyle(
        "LegalTitle",

```

```

    parent=styles["Title"],
    fontName="Helvetica-Bold",
    fontSize=16,
    leading=20,
    alignment=TA_CENTER,
    spaceAfter=20,
)
body_style = ParagraphStyle(
    "LegalBody",
    parent=styles["BodyText"],
    fontName="Times-Roman",
    fontSize=11,
    leading=16,
    spaceAfter=10,
)
story = []
# -----
# Title
# -----
story.append(
    Paragraph(
        escape(document_type.upper()),
        title_style,
    )
)
# -----
# Content
# -----
paragraphs = re.split(
    r"\n\s*\n",
    clean_text,
)
for paragraph in paragraphs:
    paragraph = paragraph.strip()
    if not paragraph:
        continue
    # Escape HTML-sensitive characters
    paragraph = escape(paragraph)
    # Preserve line breaks
    paragraph = paragraph.replace(
        "\n",
        "<br/>",
    )
    story.append(
        Paragraph(
            paragraph,
            body_style,
        )
    )
    story.append(
        Spacer(1, 4)
    )
# -----
# Build PDF

```

```
# -----  
pdf.build(story)  
output.seek(0)  
return output.getvalue()
```

TESTS

TEST_API.PY

```
from fastapi.testclient import TestClient  
import backend.routes as routes  
from backend.main import app  
class FakeGenerator:  
    model = "test-model"  
    def __init__(  
        self,  
        api_key=None,  
        model="test-model"  
    ):  
        self.model = model  
    def generate_document(  
        self,  
        document_type,  
        parties,  
        terms,  
        effective_date  
    ):  
        return (  
            f"{document_type}\n\n"  
            f"Parties: {parties}\n"  
            f"Effective Date: {effective_date}\n"  
            f"Terms: {terms}"  
        )  
def test_health():  
    client = TestClient(app)  
    response = client.get(  

```

/health

```
)  
    assert response.status_code == 200  
    assert (  
        response.json()["status"]  
        == "ok"  
    )  
def test_generate(  
    monkeypatch  
):  
    monkeypatch.setattr(  
        routes,  
        "GeminiDocumentGenerator",  
        FakeGenerator  
    )  
    client = TestClient(app)  
    response = client.post(  

```

```
"/api/generate",
json={
  "document_type": "NDA",
  "parties":
    "Jane Doe; ABC Corp",
  "terms":
```

Confidentiality;

```
"Return materials",
"effective_date":
```

2026- 09- 24

```
    }
  )
  assert response.status_code == 200
  data = response.json()
  assert (
    data["document_type"]
    == "NDA"
  )
  assert (
```

Jane Doe

```
    in data["content"]
  )
  assert (
    data["model"]
    == "test-model"
  )
def test_validation_rejects_empty_terms():
  client = TestClient(app)
  response = client.post(
    "/api/generate",
    json={
      "document_type": "NDA",
      "parties":
        "Jane Doe; ABC Corp",
      "terms": "",
      "effective_date":
```

2026- 09- 24

```
    }
  )
  assert (
    response.status_code
    == 422
  )
TEST_FORMATTER.PY
from services.document_formatter import (
  format_docx,
  format_html_preview,
```

```

format_pdf,
format_txt,
sanitize_text,
)
def test_sanitize_text():
    text = (

```

Hello\u2019s

world\u2014test \u00a0\n

```

)
result = sanitize_text(
    text
)
assert (
    result
    == "Hello's world-test"
)
def test_txt_export():
    data = format_txt(

```

Hello LegalEase

```

)
assert (
    data
    == b"Hello LegalEase"
)
def test_docx_export():
    data = format_docx(
        """
        NDA
        1. Confidentiality
        The parties agree.
        """,

```

NDA

```

)
assert data.startswith(
    b"PK"
)
def test_pdf_export():
    data = format_pdf(
        """
        NDA
        1. Confidentiality
        The parties agree.
        """,

```

NDA

```

)
assert data.startswith(

```

```

    b"%PDF"
)
def test_html_preview_escapes_markup():
    result = format_html_preview(

<script>alert(1)</script>

    )
    assert (

<script>

    not in result
    )
    assert (

&lt;script&gt;

    in result
    )

```

ENV

```

GEMINI_API_KEY=AQ.Ab8RN6Lsk2SlzSXH6OXbXuz2l4QQInylkKduG1Xw-2lZFhNPzw
GEMINI_MODEL=GEMINI_MODEL=gemini-3.6-flash
CORS_ORIGINS=http://localhost:8501,http://127.0.0.1:8501

```

.ENV.EXAMPLE

```

GEMINI_API_KEY=
GEMINI_MODEL=gemini-3.8-flash
BACKEND_URL=http://127.0.0.1:8000
CORS_ORIGINS=http://localhost:8501,http://127.0.0.1:8501
APP_NAME=LegalEase

```

.GITIGNORE

```

.venv/
__pycache__/
*.py[cod]
.pytest_cache/
.env
.streamlit/secrets.toml
*.log
.DS_Store
.vscode/

```

DOCKER- COMPOSE.YML

```

services:
  backend:
    build: .
    command:
      uvicorn backend.main:app
      --host 0.0.0.0
      --port 8000

```

```
ports:
  - "8000:8000"
env_file:
  - .env
frontend:
  build: .
  command:
    streamlit run frontend/app.py
    --server.address 0.0.0.0
    --server.port 8501
  ports:
    - "8501:8501"
  environment:
    BACKEND_URL: http://backend:8000
  env_file:
    - .env
  depends_on:
    - backend
```

DOCKERFILE

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "backend.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

PROCKFILE

```
web: uvicorn backend.main:app --host 0.0.0.0 --port ${PORT:- 8000}
```

REQUIREMENTS.TXT

```
fastapi>=0.115,<1
uvicorn[standard]>=0.30,<1
streamlit>=1.40,<2
pydantic>=2.8,<3
python-dotenv>=1.0,<2
google-genai>=1.30,<2
python-docx>=1.1,<2
fpdf2>=2.8,<3
requests>=2.32,<3
pytest>=8,<9
httpx>=0.27,<1
```